

Apprentissage Continu des Dynamiques d'un Double Pendule par World Model Hamiltonien Constraint

Axel Bröns

15 juin 2026

1 Introduction

On aperçoit le succès des World Models à travers de plusieurs papiers remarquables comme par exemple la famille des JEPA et LeWorldModel de Yann Le Cun. Le problème notable de ces modèles est l'apprentissage des dynamiques de transitions en temps discret sans contraintes physiques (qui est très important en robotique de précisions ou prévisions précises). Les trajectoires peuvent alors devenir irréalistes sur des systèmes physiques rigides. Ainsi mon approche vise à implémenter les principes fondamentaux de physique (mécanique hamiltonienne) avec la perception visuelle d'un World Model tout en modélisant de manière continu à l'aide des Neural ODEs. Dans cette étude, on va principalement se concentrer sur le double pendule (Acrobot-v1 de Gymnasium) qui est un système totalement chaotiques et non-linéaire qui est un réel challenge et cool à prédire! Le système peut très bien changer, et libre à vous de choisir votre meilleur environnement. Le but de ce papier est d'expliquer de manière claire l'intuition derrière cette approche et comprendre comment le modèle peut apprendre la physique qui nous entoure!

2 Mécanique Hamiltonienne

La première partie est là pour expliquer d'abord comment fonctionne la mécanique hamiltonienne et comment elle est intégrée dans un réseau de neurones.

2.1 Théorie

La mécanique hamiltonienne, inventée par Hamilton en 1833, est une reformulation de la mécanique classique. Elle a différent domaine d'application et a notamment permis à l'élaboration théorique de la mécanique quantique.

Mécanique Lagrangienne La mécanique hamiltonienne vient de la mécanique Lagrangienne. Concrètement les équations de mouvements d'un système à N degrés de liberté dépendent des coordonnées généralisées $\{q_i\}_{i=1,\dots,N}$ et des vitesses correspondantes, $\{\dot{q}_i\}_{i=1,\dots,N}$ où $\dot{q}_i = \frac{dq_i}{dt}$.

Le lagrangien peut donc s'écrire formellement comme une fonction : $\mathcal{L}(q_i, \dot{q}_i, t)$. les variables indexées représentant les N variables de ce type.

En mécanique hamiltonienne, le moment conjugués p_i est relié à la coordonnée généralisées q_i (en gros la position si on veut rester simple) par :

$$p_i = \frac{\partial \mathcal{L}}{\partial \dot{q}_i} \tag{1}$$

où $\mathcal{L}$ est le lagrangien et $\dot{q}_i$ est une vitesse généralisée définie comme la dérivée par rapport au temps de q_i . En coordonnées cartésiennes, les moments conjugués sont équivalents aux quantités de mouvement, alors qu'en coordonnées polaires ils correspondent aux moments angulaires.

Exemple : Le moment conjugué d'un pendule simple est :

$$p_\theta = ml^2\dot{\theta}$$

avec m la masse du pendule, l sa longueur, $\dot{\theta}$ la vitesse angulaire, $\dot{\theta} = \frac{d\theta}{dt}$, avec θ l'angle que fait le pendule avec la verticale descendante.

Mécanique Hamiltonienne Par définition, on a le hamiltonien $\mathcal{H}$ la transformée de Legendre du lagrangien :

$$\mathcal{H}(q_i, p_i, t) = \sum_k^N \dot{q}_k p_k - \mathcal{L}(q_i, \dot{q}_i, t) \quad (2)$$

Si les équations qui définissent les coordonnées généralisées sont indépendantes du temps t , on peut montrer que $\mathcal{H}$ est égal à l'énergie totale E , elle-même étant égale à la somme de l'énergie cinétique T et de l'énergie potentielle V . Alors on aura : $\mathcal{H} = E = T + V$.

Les parties qui vont nous être utiles pour notre réseau de neurones sont les équations canoniques d'Hamilton.

Sous forme différentielle $d\mathcal{H}$ et en utilisant la définition des moments conjugués donnée précédemment et les équations d'Euler Lagrange traduisant le principe de l'action minimale du lagrangien, on obtient les équations du mouvement de Hamilton, dites équations canoniques de Hamilton on retombe :

$$\frac{dq_i}{dt} = \frac{\partial \mathcal{H}}{\partial p_i} \quad \text{et} \quad \frac{dp_i}{dt} = -\frac{\partial \mathcal{H}}{\partial q_i} \quad (3)$$

On peut noter maintenant $\mathcal{S}_{\mathcal{H}} = \{\frac{\partial \mathcal{H}}{\partial p_i}, -\frac{\partial \mathcal{H}}{\partial q_i}\}$ l'évolution du temps du système. Il faut voir $\mathcal{S}_{\mathcal{H}}$ comme un champ de vecteur sur les entrées $\mathcal{H}$. On appelle ce genre de champs de vecteur les "symplectic gradient". Concrètement, ce type de champs est super important car il permet au gradient de garder un output totalement constant (et ainsi garder l'énergie totale du système constant). On peut noter aussi que les équations d'Hamilton sont des équations différentielles du premier ordre (à l'inverse des équations différentielles de Lagrange qui sont elles, d'ordre 2).

Les équations de Newton permettent également la prédiction le mouvement d'un pendule mais l'avantage d'utiliser les équations Hamiltonienne concerne le degré de liberté qui peut être généralisé à N degrés de liberté (par exemple, le corps humain possède + de 200 degrés de liberté).

Le gros avantage de travailler avec des équations différentielles est la continuité de la prédiction.

3 World Model avec HNN et Neural ODE

Concernant l'architecture d'un World Model, il se divise généralement en trois composants principaux : la vision, la mémoire, et le contrôleur [1]. Toutes les planifications et les prédictions se font dans l'espace latent.

Pour comprendre l'architecture du modèle, il faut essayer de comprendre les différentes étapes clés du modèle qui vont naturellement faire sens si vous avez compris la partie 2. On peut voir les étapes séquentiellement dans la figure 3.

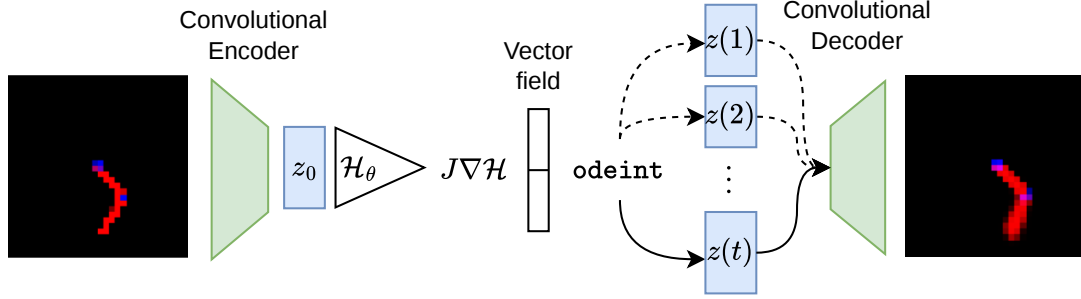


FIGURE 1 – Architecture général du modèle

1ère étape (obtention de z_0) : La première étape consiste à encoder l’information à l’aide d’un simple encodeur convolutionnel. Pour ça, on prend 3 images successives et on les passe directement dans l’encodeur. Les images arrivent en 32×32 et possèdent 9 canaux (3 images avec chaque pixels en RGB donc 3×3) et le but est de les flatten en un vecteur de dimension $d' = 1024$. (précisément on a : $(32 \times 32 \times 9) \rightarrow (16 \times 16 \times 32) \rightarrow (8 \times 8 \times 64) \rightarrow (4 \times 4 \times 128) \rightarrow (2 \times 2 \times 256) = 1024$). Maintenant on passe une simple couche linéaire pour avoir de l’espace latent $z_0 \in \mathbb{R}^{2d}$ avec $2d = 16$. Pourquoi 16 ? Parce qu’on découpe l’espace latent en deux. D’une part le vecteur position $\mathbf{q}_0$ et l’autre part, le vecteur momentum $\mathbf{p}_0$, tous les deux de dimension $\mathbb{R}^8$. On a alors juste à les concaténer pour avoir notre espace latent $z_0 = [\mathbf{q}_0, \mathbf{p}_0]^\top$.

2ème étape (obtention de $\mathcal{H}_\theta$) : Maintenant le but de cette étape est de récupérer l’énergie totale du système. Vu qu’on sait que l’énergie totale est ni plus ni moins que l’hamiltonien $\mathcal{H}$ (représente un scalaire dans l’équation 2). L’intuition sera de prendre ce scalaire pour avoir ensuite la gamme des dérivées partielles 3, permettant de faire apparaître le dt pour ensuite faire une équation diff. Pour obtenir $\mathcal{H}_\theta$ on a juste à entraîner un MLP qui prend en input $z_0 \in \mathbb{R}^{2d}$, ensuite de passer dans 2 hidden layers de dimension 200 pour ensuite finir en output de 1 (scalaire à la fin). Le tout séparé par une fonction d’activation **Tanh** pour garder au maximum la continuité (fonction de classe C^∞ donc dérivable de manière continue).

3ème étape (obtention du champs de vecteurs) : Après avoir obtenu $\mathcal{H}_\theta$, le but ici est de créer le champs de vecteurs qui dicte le déplacement de notre pendule en fonction du temps. Pour ça, nous voulons faire apparaître les équations diff (équation 3) alors nous allons simplement dériver $\mathcal{H}_\theta$ en fonction de ses paramètres q et p pour nous retrouver alors avec :

$$\nabla \mathcal{H}_\theta = \begin{bmatrix} \frac{\partial \mathcal{H}_\theta}{\partial q} \\ \frac{\partial \mathcal{H}_\theta}{\partial p} \end{bmatrix} \quad (4)$$

Le problème ici est qu’on regarde les variations de q et p et ça va regarder la pente du maximum (la tangente qui pointe toujours vers le maximum). Or nous voulons avoir une énergie constante pour représenter le plus fidèlement possible le système du pendule. Pour cela nous rajoutons une rotation vectorielle de 90 degrés qui est représenté par cette matrice symplectic :

$$\mathbf{J} = \begin{pmatrix} \mathbf{0}_d & \mathbf{I}_d \\ -\mathbf{I}_d & \mathbf{0}_d \end{pmatrix} \quad (5)$$

avec $\mathbf{I}_d$ la matrice identité (de dimension $d \times d$). Cette matrice nous permet de garder notre vecteur à “plat” et garder une énergie constante au fil du temps. Au final nous avons :

$$\dot{z} = \frac{dz}{dt} = \mathbf{J} \nabla \mathcal{H}_\theta = \begin{bmatrix} \frac{\partial \mathcal{H}_\theta}{\partial q} \\ -\frac{\partial \mathcal{H}_\theta}{\partial p} \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{q}} \\ -\dot{\mathbf{p}} \end{bmatrix} \quad (6)$$

Magie magie! On retrouve les bonnes équations différentielles comme ici 3 et nous avons un champ de vecteur qui décrit le mouvement du pendule en fonction du temps.

4ème étape (obtention de $z(t)$) : Cette étape consiste seulement à récupérer correctement la position du pendule de manière continu $z(t)$. Dans le code on applique un simple solver d’ODE basique de type `euler` ou `rk4` qui vient résoudre pour un t choisis :

$$z_{t+1} = z_t + \int_t^{t+\Delta t} \frac{dz}{dt} dt \approx z_t + \Delta t \cdot \begin{bmatrix} \frac{\partial H}{\partial p} \\ -\frac{\partial H}{\partial q} \end{bmatrix} \quad (7)$$

Ainsi on peut avoir facilement n’importe quelle position à un t choisis ce qui assure la continuité dans les prédictions.

5ème étape (reconstruction de notre z_{t+1}) : Ici, il suffit de prendre l’état de l’espace latent prédit $z(t + \Delta T)$, avec ΔT qu’on peut choisir librement, et ensuite de le reconstruire pour permettre de le voir directement en 2D (32×32). On remarque ici que notre $z(t + \Delta T) \in \mathbb{R}^{2d}$ est de dimension $2d$, pour le rescale et le decoder facilement on le passe dans un MLP avec input $2d$ et 2 hidden layers de dimension 200 et pour finir un output de dimension $2 \times 2 \times 256$. Symétriquement à la première étape, il suffit de reconstruire avec un décodeur convolutionnel. ($1024 = (2 \times 2 \times 256) \rightarrow (4 \times 4 \times 128) \rightarrow (8 \times 8 \times 64) \rightarrow (16 \times 16 \times 32) \rightarrow (32 \times 32 \times 16) \rightarrow (32 \times 32 \times 3)$)

4 Expérimentations

Cette section détaille le protocole expérimental, le traitement des données, ainsi que les résultats qualitatifs et quantitatifs obtenus.

4.1 Génération du Dataset et Prétraitement

Concernant le jeu de données, j’ai utilisé l’environnement `Acrobot-v1` de la bibliothèque Gymnasium d’OpenAI. Le choix de ce système s’explique par la nature hautement chaotique et non-linéaire de la dynamique du double pendule passif, ce qui constitue un défi majeur pour la prédiction temporelle continue.

Prétraitement Avant de commencer l’entraînement, plusieurs transformations sont appliquées aux images générées par le double pendule afin de faciliter l’entraînement de l’encodeur convolutionnel. Premièrement, les couleurs de l’image sont inversées pour obtenir un arrière-plan à prédominance noire (valeurs proches de 0.0) et concentrer l’information sur le pendule lui-même. Deuxièmement, afin d’éliminer le bruit résiduel et d’accentuer la parcimonie du signal d’entrée, un filtre de seuillage est appliqué : tous les pixels d’intensité inférieure à 0.2 sont ramenés à 0.0.

Entraînement Concernant l’entraînement, j’ai choisis N séquences (aux alentours de 1500-2000 séquences), toutes avec $T = 10$ frames de taille 32×32 avec 3 canaux de couleur (RGB). Le but ici, est vraiment d’avoir une gamme diverse du mouvement de pendule pour qu’il puisse apprendre au mieux et ne pas apprendre un pendule statique. Pour permettre cette diversité, j’applique une action constante neutre (couple nul, correspondant à l’action 1 de l’environnement) à chaque pas de temps.

4.2 Objectifs d’entraînement et régularisations physiques

Le modèle est entraîné de bout en bout en utilisant l’optimiseur Adam avec un taux d’apprentissage de 10^{-3} sur un total de E époques (généralement aux alentours de 40-180 epochs) avec une

taille de lot (batch size) de 16. La loss, est un mix de plusieurs loss qui vont agir sur différentes notions différentes. Le but sera de pénaliser : une mauvaise reconstruction visuelle, la physique non respectée et la norme de notre espace latent non respectée. Et ainsi notre fonction loss est comme suit :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \lambda_{\text{phys}} (\mathcal{L}_{\text{cc}} + \mathcal{L}_{\text{energy}}) \quad (8)$$

Perte de reconstruction ($\mathcal{L}_{\text{recon}}$) : J'applique simplement une perte d'entropie croisée binaire avec logits (`BCEWithLogitsLoss`). Concrètement, elle force l'encodeur à extraire les informations importante pour pouvoir construire le meilleur espace latent possible. Dorénavant, si nous mettons cette loss comme ça, le modèle va complètement tricher et va mettre l'image toute en noir et va avoir raison 95% du temps car l'image comporte plus de pixels noirs que de pixels important. Pour cela, je mets `pos.weight=5.0` pour mettre plus d'importance sur les pixels non noirs (les vrais informations importantes du pendule pour notre pendule).

Perte de conservation de l'énergie ($\mathcal{L}_{\text{energy}}$) : Cette loss est essentielle, comme on l'a vu, l'énergie totale ($\mathcal{H}_\theta$) doit être complètement constant pour respecter la physique de notre monde et ainsi être représentatif. Formellement, elle est décrite comme ça :

$$\mathcal{L}_{\text{energy}} = \frac{1}{T-2} \sum_{t=2}^T (H_\theta(z_t) - H_\theta(z_{t-1}))^2 \quad (9)$$

L'intuition est de pénaliser le modèle dès que la différence de l'énergie (l'hamiltonien) devient trop importante entre deux position de l'espace latent.

Perte de cohérence de coordonnées ($\mathcal{L}_{\text{cc}}$) : Avec cette perte, il faut bien faire comprendre au modèle comment se comporte notre espace latent qui a une hierarchie bien définie ($z = [\mathbf{q}, \mathbf{p}]^\top$) avec les premiers vecteurs correspondant aux positions et la deuxième partie correspondant aux moments de notre pendule. Pour qu'il apprenne correctement cette dynamique, on va appliquer une perte de cohérence de coordonnées, formellement elle est décrite comme suit :

$$\mathcal{L}_{\text{cc}} = \frac{1}{T-3} \sum_{t=3}^T \left\| p_{t-1} - \frac{q_t - q_{t-1}}{\Delta t} \right\|^2 \quad (10)$$

Concrètement, la formule décrit l'écart entre la prédiction théorique et la réalité du déplacement.

Pour éviter les contraintes physiques ne viennent perturber la phase initiale de la représentation visuelle, on ajoute un coefficient $\lambda_{\text{phys}} = 0$ pour les 5 premières époques, puis $\lambda_{\text{phys}} = 0.1$ jusqu'à la fin de l'entraînement.

4.3 Résultats

Les résultats qualitatifs démontrent la capacité du modèle à projeter les observations visuelles dans l'espace des phases et à prédire les frames futures par intégration temporelle continue. Bien évidemment, on peut prédire à n'importe quel timeframe T . Voici les résultats d'un entraînement de 80 epochs sur 2000 séquences. La première ligne correspond au Ground Truth (le vrai pendule) et la ligne d'en bas correspond à la prédiction de mon modèle.

On note l'underfitting léger car il n'a pas réussi à totalement représenter le pendule de manière nette. Une des améliorations pour contrer ce problème serait de l'entraîner plus longtemps.

Pour analyser plus précisément le comportement géométrique, la Figure 3 illustre le tracé continu de l'extrémité du double pendule dans l'espace image. La trajectoire prédite par l'intégration

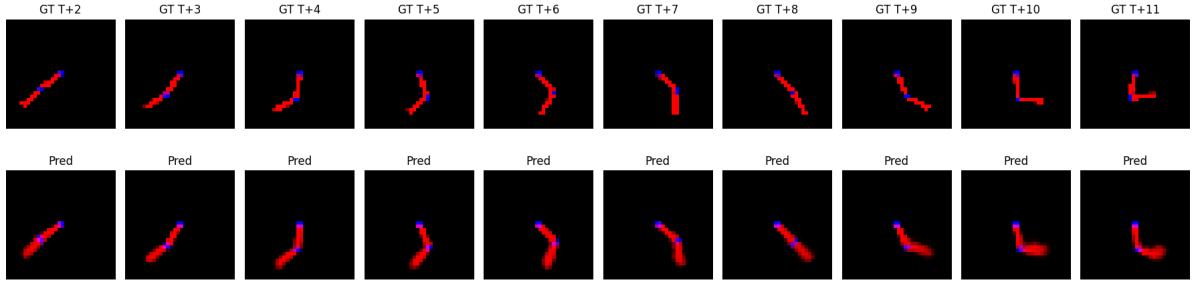


FIGURE 2 – Comparaison qualitative temporelle entre les observations réelles (GT, ligne du haut) et les prédictions générées par le décodeur du World Model à partir de la trajectoire latente intégrée par le HNN-ODE (Pred, ligne du bas) pour les pas de temps $T + 2$ à $T + 11$.

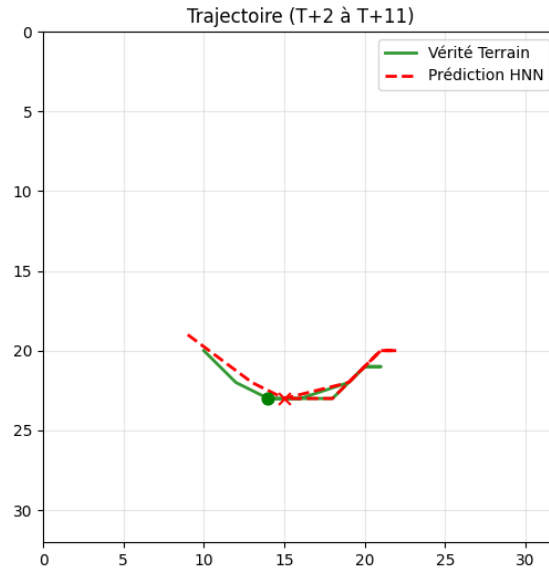


FIGURE 3 – Visualisation continue de la trajectoire réelle (ligne verte continue) et prédite par le HNN (ligne rouge pointillée) de l'extrémité du double pendule dans l'espace image de $T + 2$ à $T + 11$.

continue du Hamiltonien appris suit fidèlement la trajectoire réelle, démontrant que l’espace des phases latent conserve les propriétés topologiques fondamentales de l’Acrobot.

Pour évaluer quantitativement la conservation de la physique, la Figure 4 compare l’évolution temporelle de l’énergie physique totale du système sur un long horizon de prédiction (40 étapes de temps de $\Delta t = 0.2$ s) pour la vérité terrain (Acrobot passif), notre modèle (HNN-ODE) et une baseline (Neural ODE classique sans contrainte hamiltonienne).

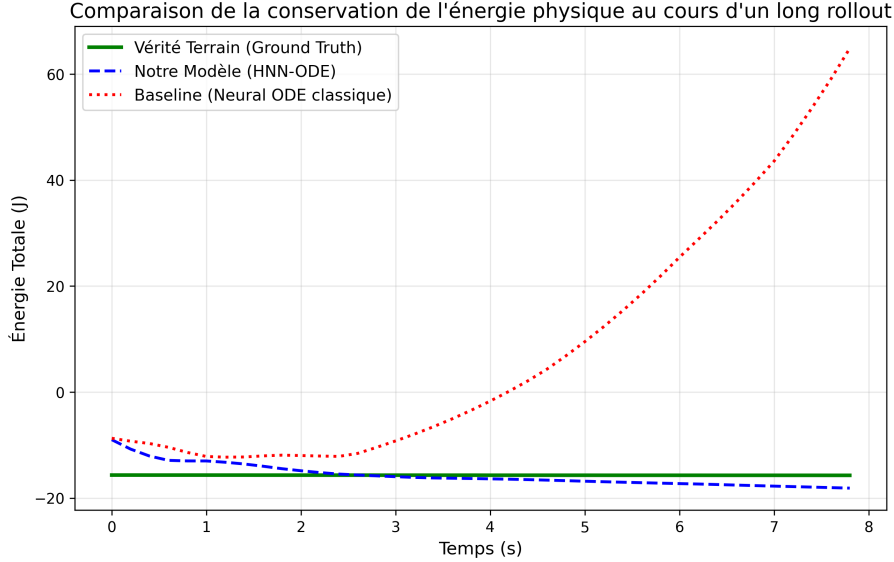


FIGURE 4 – Évolution temporelle de l’énergie physique totale du système. La vérité terrain (ligne verte) reste parfaitement constante. Notre modèle HNN-ODE (ligne bleue en pointillés) préserve la conservation de l’énergie avec des oscillations infimes, tandis que la baseline Neural ODE classique (ligne rouge pointillée) dissipe artificiellement l’énergie, illustrant l’importance de la régularisation hamiltonienne.

On constate que la baseline standard dévie rapidement du comportement physique réel, entraînant une dissipation prononcée de l’énergie au cours du temps. En revanche, le modèle HNN-ODE reste extrêmement proche de l’énergie de la vérité terrain, validant l’efficacité de la structure symplectique induite et de la fonction de perte physique $\mathcal{L}_{\text{energy}}$ et $\mathcal{L}_{\text{cc}}$.

5 Conclusion

Pour conclure, comme on a vu à travers ce papier, on peut très bien avoir un world model pouvant apprendre la physique qui nous entoure tout en restant dans son espace latent. On peut aussi apercevoir l’importance du choix des loss du modèle qui ont un impact considérable sur l’entraînement de celui-ci et donc de ses performances finales. Pour plus tard, on pourrait très bien ajouter la partie “controller” du world model pour le model puisse interagir avec le monde qui nous entoure. Ce travail ouvre de nouvelles perspectives pour le développement de world model plus robuste et physiquement réalistes pour la planification et le contrôle en robotique.

Références

- [1] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems 31*, pages 2451–2463. Curran Associates, Inc., 2018.